CSC 8607 – Introduction au deep learning

CI3 : Réseaux convolutifs

Objectifs : *Dans ce TP, vous allez implémenter et analyser différents réseaux de neurones (MLP vs CNN) pour résoudre un problème de classification d'images à grande échelle sur le jeu de données* ***ImageNet****. Vous étudierez l'impact de l'augmentation de données, du Dropout, de l'Early Stopping, ainsi que l'empreinte mémoire (VRAM) des couches convolutives par rapport aux couches denses.*

Modalités de rendu : *L'évaluation se fera sur la base d'un rapport au format Markdown (*`rapport.md`*). Dans le même dépôt Git que lors des TPs précédents, créez un répertoire* `TP3` *et placez-y votre rapport, vos scripts Python et votre script* `sbatch`*. Mettez à jour votre dépôt régulièrement.*

Remarque : Gestion du temps de calcul sur ImageNet : *ImageNet contient plus d'un million d'images. Une seule époque sur l'ensemble du jeu de données prend beaucoup de temps. Ce TP est donc conçu en deux vitesses :*

- *Vous validerez votre code et ferez vos premières comparaisons en séance sur un* ***sous-ensemble réduit*** *d'ImageNet afin d'obtenir des résultats en quelques minutes.*
- *En fin de séance, vous lancerez un* ***entraînement long*** *en tâche de fond via* `sbatch`*. Même si cet entraînement ne termine pas ses 100 époques avant votre rendu, les questions portent sur l'observation des premières heures d'apprentissage.*

Objectifs

- Charger et prétraiter le dataset ImageNet avec augmentation de données et normalisation.
- Comparer théoriquement et empiriquement un Perceptron Multicouche (MLP) et l'architecture convolutive AlexNet.
- Calculer les dimensions spatiales, le nombre de paramètres et l'empreinte mémoire des activations (cartes de caractéristiques).
- Mettre en place l'*Early Stopping*.
- Soumettre, surveiller et analyser un entraînement long sur cluster GPU via `sbatch`.

Exercice 1 : Exercice 1 : Préparation d'ImageNet et Augmentation de données (~30mn)

Le jeu de données ImageNet (ILSVRC 2012) est stocké localement sur les nœuds du cluster dans le dossier `/mnt/ssd/imagenet/2012`. Il comporte 1000 classes. Comme le jeu de test officiel n'a pas d'étiquettes publiques, nous utiliserons le split `val` pour la validation et l'évaluation finale.

Question 1.a. Créez un fichier `train_imagenet.py` dans votre dossier `TP3`. Complétez le code à trous ci-dessous pour définir les pipelines de prétraitement et d'augmentation de données :

- **Pour l'entraînement (augmentation aléatoire) :** un redimensionnement et recadrage aléatoire (`RandomResizedCrop`) de taille `227`, suivi d'un miroir horizontal aléatoire (`RandomHorizontalFlip`), de la conversion en tenseur et de la normalisation.
- **Pour la validation (déterministe) :** un redimensionnement (`Resize`) à `256`, un découpage centré (`CenterCrop`) de taille `227`, suivi de la conversion en tenseur et de la normalisation.

```
import os
import torch
```

```python
import torch.nn as nn
import torch.optim as optim
from torchvision import transforms
from torchvision.datasets import ImageNet
from torch.utils.data import DataLoader, Subset
from torch.utils.tensorboard import SummaryWriter
from tqdm import tqdm

data_dir = "/mnt/ssd/imagenet/2012"
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD  = [0.229, 0.224, 0.225]

# 1. Pipeline d'entraînement (avec Data Augmentation)
train_transform = transforms.Compose([
    transforms._________________(227),
    transforms._________________(),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
])

# 2. Pipeline de validation (déterministe)
val_transform = transforms.Compose([
    transforms.Resize(256),
    transforms._________(227),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
])

def get_dataloaders(batch_size=256, n_workers=8, subset_size=None):
    print("Chargement d'ImageNet...")
    train_dataset = ImageNet(root=data_dir, split="train", transform=_________________)
    val_dataset   = ImageNet(root=data_dir, split="val",   transform=_________________)

    # Optionnel : sous-ensemble pour tester rapidement le code en séance
    if subset_size is not None:
         generator = torch.Generator().manual_seed(42)
         train_indices = torch.randperm(len(train_dataset), generator=generator)[:subset_size]
         val_indices   = torch.randperm(len(val_dataset), generator=generator)[:subset_size // 4]
         train_dataset = Subset(train_dataset, train_indices)
         val_dataset   = Subset(val_dataset, val_indices)

    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=_____,
num_workers=n_workers, pin_memory=True)
    val_loader   = DataLoader(val_dataset,   batch_size=batch_size, shuffle=_____,
num_workers=n_workers, pin_memory=True)
    return train_loader, val_loader
```

Remarque : Lire et décoder des centaines de milliers d'images JPEG sur le disque prend du temps CPU. Le paramètre **`num_workers=8`** *parallélise ce chargement. Pensez à demander 8 CPUs à Slurm (*`--cpus-per-task=8`*) lorsque vous lancerez vos scripts.*

✍️ À mettre dans votre rapport (`rapport.md`) :

1. Que se passe-t-il mathématiquement et numériquement lors de la descente de gradient si l'on oublie l'étape **`transforms.Normalize`** ? Pourquoi les statistiques (moyenne et écart-type) doivent-elles être calculées uniquement sur le jeu d'entraînement ?
2. Pourquoi utilise-t-on **`RandomResizedCrop`** et **`RandomHorizontalFlip`** sur le jeu d'entraînement, mais **`Resize`** + **`CenterCrop`** (sans flip) sur le jeu de validation ?
3. Donnez un exemple de tâche de classification d'images où l'augmentation par **`RandomHorizontalFlip`** serait néfaste et fausserait les étiquettes.

Exercice 2 : Exercice 2 : Étude théorique et implémentation des architectures (MLP vs AlexNet) (~45mn)

Avant de lancer les entraînements, nous allons comparer l'architecture d'un Perceptron Multicouche (MLP) avec celle d'AlexNet (Krizhevsky et al., 2012), le réseau convolutif historique qui a popularisé le Deep Learning sur ImageNet.

Étape 1 : Implémentation du MLP (Baseline)

Question 2.a. Complétez la classe **`MLP`** ci-dessous comportant deux couches cachées de taille **`256`** activées par **`ReLU`**, prenant en entrée nos images de taille $3 \times 227 \times 227$ et prédisant 1000 classes.

```
class MLP(nn.Module):
    def __init__(self, input_size=3 * 227 * 227, hidden_size=256, num_classes=1000):
        super().__init__()
        self.classifier = nn.Sequential(
            nn.Linear(__________, hidden_size),
            nn.ReLU(inplace=True),
            nn.Linear(hidden_size, ___________),
            nn.ReLU(inplace=True),
            nn.Linear(hidden_size, ___________),
        )

    def forward(self, x):
        x = torch.flatten(x, 1)
        return self.classifier(x)
```

✍️ À mettre dans votre rapport :

1. Calculez à la main le nombre exact de paramètres de ce MLP (en comptant les biais, puis sans les biais). Quelle couche concentre plus de 99% des paramètres ?
2. Outre ce nombre élevé de paramètres en entrée, quel est le défaut structurel majeur du MLP lorsqu'on lui fournit une image aplatie (**`flatten`**) ?

Étape 2 : Analyse et implémentation d'AlexNet

Question 2.b. Voici la description détaillée de l'architecture AlexNet pour une image d'entrée de taille $3 \times 227 \times 227$:

Couche	Taille entrée (C, H×W)		Hyperparamètres (Filtres/Neurones, K, S, P)				Taille sortie (C, H×W)		# Paramètres (avec biais)	Mémoire sortie (KB)
Conv1 + ReLU	3	227×227	64	11×11	S=4	P=2	?	?	?	?
MaxPool1	?	?	-	3×3	S=2	P=0	?	?	0	?
Conv2 + ReLU	?	?	192	5×5	S=1	P=2	?	?	?	?
MaxPool2	?	?	-	3×3	S=2	P=0	?	?	0	?
Conv3 + ReLU	?	?	384	3×3	S=1	P=1	?	?	?	?
Conv4 + ReLU	?	?	256	3×3	S=1	P=1	?	?	?	?
Conv5 + ReLU	?	?	256	3×3	S=1	P=1	?	?	?	?
MaxPool3	?	?	-	3×3	S=2	P=0	?	?	0	?
Flatten + Dropout	?	?	p = 0.5				? (1D)		0	?
Linear1 + ReLU + Dropout	?		4096 neurones (p = 0.5)				4096		?	?
Linear2 + ReLU	4096		4096 neurones				4096		?	?
Linear3 (Sortie)	4096		1000 classes				1000		?	?

✍️ À mettre dans votre rapport :

1. Recopiez et complétez l'intégralité de ce tableau en format Markdown dans votre rapport. Pour la colonne **Mémoire sortie (KB)**, calculez la mémoire occupée par les activations de sortie de chaque couche pour **une seule image** en supposant des flottants 32 bits (4 octets, $1\text{ KB} = 1024\text{ octets}$).
2. Dans AlexNet, où se concentre l'essentiel des **paramètres** (poids) ? À l'inverse, où se concentre l'essentiel de la **mémoire des activations** (indispensable à stocker lors du *forward* pour calculer la *backpropagation*) ?
3. Comparez la mémoire totale des activations d'AlexNet (environ 2340 KB par image) avec celle du MLP de l'étape 1. Expliquez ce paradoxe alors que les deux réseaux ont un ordre de grandeur similaire en nombre de paramètres.

Question 2.c. Complétez maintenant l'implémentation PyTorch d'**`AlexNet`** et la fonction de vérification du nombre de paramètres dans votre script :

```
class AlexNet(nn.Module):
    def __init__(self, num_classes=1000):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(64, 192, kernel_size=___, padding=___),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
```

```
            nn.Conv2d(192, 384, kernel_size=___, padding=___),
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=___, padding=___),
            nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=___, padding=___),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
        )
        self.classifier = nn.Sequential(
            nn.Dropout(p=0.5),
            nn.Linear(256 * ___ * ___, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(p=0.5),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Linear(4096, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x

def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)
```

✍️ **Action :** Affichez le résultat de **`count_parameters`** pour **`MLP()`** et **`AlexNet()`** afin de vérifier vos calculs théoriques de la question précédente.

Exercice 3 : Exercice 3 : Boucle d'entraînement, Early Stopping et Comparaison rapide (~45mn)

Nous allons maintenant assembler la boucle d'entraînement instrumentée avec **TensorBoard** et dotée d'un mécanisme d'**Early Stopping** pour éviter de gaspiller du temps GPU lorsque le modèle n'apprend plus.

Question 3.a. Complétez la fonction d'évaluation et la boucle principale incluant l'Early Stopping (patience de 5 époques) :

```
@torch.no_grad()
def evaluate(loader, model, criterion):
    model.______() # Désactive le Dropout !
    running_loss, correct, total = 0.0, 0, 0
    for images, labels in tqdm(loader, desc="Eval", leave=False):
        images, labels = images.to(device, non_blocking=True), labels.to(device, non_blocking=True)
        outputs = model(images)
        loss = criterion(outputs, labels)
        running_loss += loss.item() * images.size(0)
        _, predicted = outputs.max(1)
        correct += (predicted == labels).sum().item()
        total += labels.size(0)
```

```
    return running_loss / total, correct / total


def train_model(model_name="alexnet", batch_size=256, lr=1e-4, num_epochs=20, weight_decay=1e-5,
patience=5, subset_size=20000):
    train_loader, val_loader = get_dataloaders(batch_size=batch_size, subset_size=subset_size)

    if model_name == "alexnet":
        model = AlexNet(num_classes=1000).to(device)
    elif model_name == "mlp":
        model = MLP(num_classes=1000).to(device)
    else:
        raise ValueError("Modèle inconnu")

    print(f"Modèle {model_name} initialisé avec {count_parameters(model):,} paramètres.")

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)

    run_ tag = f"{model_name}_sub{subset_size}_lr{lr}"
    writer = SummaryWriter(log_dir=f"runs/{run_tag}")

    best_val_loss = float('inf')
    early_stop_counter = 0

    for epoch in range(num_epochs):
        model._______() # Active le Dropout !
        running_loss, correct_train, total_train = 0.0, 0, 0

        for images, labels in tqdm(train_loader, desc=f"Epoch {epoch+1}/{num_epochs}"):
            images, labels = images.to(device, non_blocking=True), labels.to(device, non_blocking=True)
            optimizer.zero_grad(set_to_none=True)
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * images.size(0)
            _, predicted = outputs.max(1)
            correct_train += (predicted == labels).sum().item()
            total_train += labels.size(0)

        train_loss = running_loss / total_train
        train_acc  = correct_train / total_train
        val_loss, val_acc = evaluate(val_loader, model, criterion)

        writer.add_scalar('Loss/Train', train_loss, epoch)
        writer.add_scalar('Loss/Validation', val_loss, epoch)
```

```
        writer.add_scalar('Accuracy/Train', train_acc, epoch)
        writer.add_scalar('Accuracy/Validation', val_acc, epoch)
        writer.flush()

        print(f"Epoch [{epoch+1}/{num_epochs}] | Train Loss: {train_loss:.4f} (Acc: {train_acc:.4f}) |
Val Loss: {val_loss:.4f} (Acc: {val_acc:.4f})")

        # --- EARLY STOPPING ---
        if val_loss = __________:
                print("Early stopping déclenché !")
                break

    writer.close()
```

✍️ **Action en séance (Mode Rapide) :** Lancez successivement un entraînement de **10 époques** sur le sous-ensemble (**`subset_size=20000`**) pour **`mlp`** puis pour **`alexnet`**.

✍️ **À mettre dans votre rapport :**

1. Quel est le rôle exact de **`model.eval()`** et **`model.train()`** vis-à-vis des couches **`nn.Dropout`** présentes dans AlexNet ? Que se passerait-il si l'on oubliait **`model.eval()`** pendant l'évaluation ?
2. Sur un jeu de données à 1000 classes équilibrées comme ImageNet, quelle est la valeur théorique de l'accuracy et de la **`CrossEntropyLoss`** ($\ln(1000)$) à l'initialisation (choix purement aléatoire) ? Vos premières époques sont-elles cohérentes avec ces valeurs ?
3. Comparez les courbes TensorBoard (Train/Val Loss et Accuracy) du MLP et d'AlexNet sur ce sous-ensemble de 20 000 images. Lequel apprend le mieux ? Lequel sur-apprend (*overfit*) le plus vite et pourquoi ? Insérez une capture d'écran de vos courbes TensorBoard dans le rapport.

Étape 2 : Impact de la profondeur (Modification d'architecture)

Question 3.b. Créez une variante d'AlexNet dans laquelle vous **ajoutez 3 couches de convolution supplémentaires** (de taille $256 \times 3 \times 3$, padding 1, suivies de ReLU) juste après **`Conv5`**, et lancez un test rapide sur 10 époques avec **`subset_size=20000`**.

✍️ **À mettre dans votre rapport :** Observez les courbes d'entraînement de ce réseau plus profond par rapport à l'AlexNet standard. La dégradation ou la difficulté à démarrer l'apprentissage que vous observez au début relève-t-elle du **sur-apprentissage (overfitting)** ou du **sous-apprentissage (underfitting)** ? Comment l'expliquez-vous du point de vue de la propagation du gradient ?

Exercice 4 : Exercice 4 : Lancement et analyse d'un entraînement long via sbatch (~30mn + suivi)

En Deep Learning, on ne reste pas devant son terminal à attendre qu'un entraînement sur ImageNet se termine. Vous allez maintenant lancer un entraînement d'**AlexNet sur l'intégralité d'ImageNet** (**`subset_size=None`**) en tâche de fond via **`sbatch`**.

Remarque : Important pour le rendu : *Vous n'avez **pas** besoin d'attendre que les 100 époques soient terminées pour rendre votre rapport ! Laissez tourner votre job pendant au moins **1h à 2h** (ou quelques époques complètes), puis répondez aux questions ci-dessous à partir des logs et des courbes TensorBoard générées jusque-là.*

Question 4.a. Modifiez l'appel dans votre bloc **`if __name__ == "__main__":`** de **`train_imagenet.py`** pour lancer AlexNet avec **`subset_size=None`** et **`num_epochs=50`**. Créez ensuite un fichier **`run_imagenet.sh`** dans votre dossier **`TP3`** en complétant ce script Slurm :

```
#!/bin/bash
#SBATCH --partition=gpu
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=8
#SBATCH --mem=24G
#SBATCH --time=04:00:00
#SBATCH -J alexnet-full
#SBATCH -o logs/%x-%j.out
#SBATCH -e logs/%x-%j.err

set -euo pipefail
mkdir -p logs

# Activation de l'environnement virtuel Mamba/Conda
source ~/miniforge3/etc/profile.d/conda.sh
conda activate deeplearning

echo "Début du job $SLURM_JOB_ID sur le nœud $SLURM_NODELIST"
nvidia-smi

python -u train_imagenet.py
```

✍️ Action : Soumettez votre job avec **`sbatch run_imagenet.sh`** et vérifiez avec **`squeue -u $USER`** qu'il passe bien à l'état **`R`** (Running).

✍️ À mettre dans votre rapport :

1. Profilage temporel et matériel : En consultant votre fichier de sortie **`logs/alexnet-full-XXXX.out`** (via **`tail -f`** ou **`less`**), indiquez :
   - Combien de temps (en minutes/secondes) dure exactement une époque complète sur les 1,28 million d'images d'entraînement avec votre GPU ?
   - À cette vitesse, combien d'heures faudrait-il pour compléter 100 époques sans Early Stopping ?
2. Analyse de l'entraînement au fil de l'eau : Après avoir laissé tourner le job pendant au moins 3 à 5 époques complètes (ou ~1h30), rapatriez le dossier **`runs/`** et **`logs/`**.
   - Insérez dans votre rapport la courbe TensorBoard (ou le tableau des valeurs par époque) de **`Loss/Train`**, **`Loss/Validation`**, **`Accuracy/Train`** et **`Accuracy/Validation`** obtenues sur le dataset complet.
   - Comparez l'écart entre **`Loss/Train`** et **`Loss/Validation`** sur le dataset complet (1,28M d'images + Data Augmentation) avec celui que vous aviez observé à l'exercice 3 sur le petit sous-ensemble de 20 000 images. Pourquoi le sur-apprentissage est-il beaucoup plus faible ici ?
3. Observation surprenante : Lors des premières époques, il arrive fréquemment que la perte de validation (**`Loss/Validation`**) soit légèrement inférieure (meilleure) à la perte d'entraînement (**`Loss/Train`**). Donnez deux raisons techniques liées à notre code (indice : regardez ce qui est actif pendant le **`train`** mais désactivé pendant le **`val`**) qui expliquent ce phénomène !

Remarque : N'oubliez pas de commiter vos dernières modifications (`rapport.md`, scripts Python, script `.sh` et captures TensorBoard) dans votre dossier `TP3`, et de faire un push sur votre dépôt Git.